```
<!DOCTYPE html> <html lang="ko"> <head> <meta charset="UTF-8" /> <meta
name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0,
user-scalable=no, viewport-fit=cover" /> <title>체리 (Cherry) - 나만의 영어 지문 암기실</title>
<!-- PWA & Mobile Web App Meta Tags --> <meta name="theme-color" content="#e11d48" />
<meta name="apple-mobile-web-app-capable" content="yes" /> <meta
name="apple-mobile-web-app-status-bar-style" content="black-translucent" /> <meta
name="apple-mobile-web-app-title" content="체리 암기" />
<!-- Web App Manifest (Dynamic Link) --> <link id="pwa-manifest" rel="manifest" href="" />
<!-- Tailwind CSS CDN --> <script src="https://cdn.tailwindcss.com"></script> <script>
tailwind.config = { darkMode: 'class', theme: { extend: { colors: { cherry: { 50: '#fff1f2', 100:
'#ffe4e6', 200: '#fecdd3', 300: '#fda4af', 400: '#fb7185', 500: '#f43f5e', 600: '#e11d48', 700:
'#be123c', 800: '#9f1239', 900: '#881337', 950: '#4c0519', }, darkbg: '#121316', darkcard:
'#1c1e24' } } } } </script>
<!-- FontAwesome Icons --> <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css" /> <!-- React &
Babel CDNs --> <script
src="https://cdnjs.cloudflare.com/ajax/libs/react/18.2.0/umd/react.production.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/react-dom/18.2.0/umd/react-dom.production.min.js"><
/script> <script
src="https://cdnjs.cloudflare.com/ajax/libs/babel-standalone/7.23.6/babel.min.js"></script>
<!-- Google Pretendard Font --> <link rel="stylesheet" as="style" crossorigin
href="https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/static/pretendard.min.c
ss" />
<!-- Firebase Modular SDK for Cloud Auth & Firestore Database Sync --> <script
type="module"> import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-app.js"; import { getAuth,
GoogleAuthProvider, signInWithPopup, signInWithEmailAndPassword,
createUserWithEmailAndPassword, signOut, onAuthStateChanged } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-auth.js"; import { getFirestore, doc, setDoc,
deleteDoc, collection, query, where, onSnapshot, getDocs } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-firestore.js";
const appId = typeof __app_id !== 'undefined' ? __app_id : 'cherry-app-v1'; const firebaseConfig
= typeof __firebase_config !== 'undefined' ? JSON.parse(__firebase_config) : null;
let app, auth, db; if (firebaseConfig) { try { app = initializeApp(firebaseConfig); auth =
getAuth(app); db = getFirestore(app); } catch (e) { console.warn("Firebase 연결 알림:", e); } }
window.CherryFirebase = { appId, auth, db, GoogleAuthProvider, signInWithPopup,
signInWithEmailAndPassword, createUserWithEmailAndPassword, signOut,
onAuthStateChanged, doc, setDoc, deleteDoc, collection, query, where, onSnapshot, getDocs };
</script>
<!-- Dynamic PWA Manifest & Service Worker Script --> <script> (function() { const
manifestJson = { name: "체리 - 영어 지문 암기 웹앱", short_name: "체리", start_url: "./", display:
"standalone", background_color: "#121316", theme_color: "#e11d48", icons: [ { src:
"https://placehold.co/192x192/e11d48/ffffff?text=🍒", sizes: "192x192", type: "image/png" }, { src:
"https://placehold.co/512x512/e11d48/ffffff?text=🍒", sizes: "512x512", type: "image/png" } ] };
const blob = new Blob([JSON.stringify(manifestJson)], {type: 'application/json'}); const
manifestUrl = URL.createObjectURL(blob); document.addEventListener('DOMContentLoaded',
() => { const manifestEl = document.getElementById('pwa-manifest'); if (manifestEl)
```

```
manifestEl.setAttribute('href', manifestUrl); });
if ('serviceWorker' in navigator) { const swCode = self.addEventListener('install', (e) =>
self.skipWaiting()); self.addEventListener('activate', (e) => e.waitUntil(clients.claim()));
self.addEventListener('fetch', (e) => e.respondWith(fetch(e.request).catch(() =>
caches.match(e.request))));; const swBlob = new Blob([swCode], { type: 'text/javascript' });
navigator.serviceWorker.register(URL.createObjectURL(swBlob)).catch(() => {}); } })(); </script>
<style> * { -webkit-tap-highlight-color: transparent; } body { font-family: 'Pretendard',
-apple-system, BlinkMacSystemFont, system-ui, Roboto, sans-serif; padding-bottom:
env(safe-area-inset-bottom); } .no-scrollbar::-webkit-scrollbar { display: none; } .no-scrollbar {
-ms-overflow-style: none; scrollbar-width: none; } .touch-canvas { touch-action: none;
-webkit-user-select: none; user-select: none; }
@media print { .no-print { display: none !important; } .print-only { display: block !important; } body
{ background: white !important; color: black !important; } } @media screen { .print-only { display:
none !important; } } </style> </head> <body class="bg-stone-50 dark:bg-darkbg text-stone-900
dark:text-stone-100 font-sans min-h-screen transition-colors duration-200 overflow-x-hidden">
<div id="root"></div>
<script type="text/babel"> const { useState, useEffect, useRef, useMemo, useCallback } =
React;
// ========================================== // 1. Storage Engine (CherryDB -
IndexedDB + LocalStorage Dual Sync) //
========================================== const DB_NAME =
'CherryMemorizeDB'; const DB_VERSION = 1; const LOCAL_STORAGE_PASSAGES =
'cherry_passages_v1'; const LOCAL_STORAGE_SETTINGS = 'cherry_settings_v1'; const
LOCAL_STORAGE_CURRENT_ID = 'cherry_current_id_v1'; const LOCAL_STORAGE_TAB =
'cherry_tab_v1';
const CherryDB = { open() { return new Promise((resolve, reject) => { try { const req =
indexedDB.open(DB_NAME, DB_VERSION); req.onupgradeneeded = (e) => { const db =
e.target.result; if (!db.objectStoreNames.contains('passages')) db.createObjectStore('passages',
{ keyPath: 'id' }); if (!db.objectStoreNames.contains('settings')) db.createObjectStore('settings', {
keyPath: 'key' }); }; req.onsuccess = () => resolve(req.result); req.onerror = (err) => reject(err); }
catch (err) { reject(err); } }); }, async getAllPassages() { let dbPassages = []; try { const db =
await this.open(); dbPassages = await new Promise((resolve, reject) => { const tx =
db.transaction('passages', 'readonly'); const store = tx.objectStore('passages'); const req =
store.getAll(); req.onsuccess = () => resolve(req.result || []); req.onerror = (err) => reject(err); });
} catch (e) {}
if (dbPassages && dbPassages.length > 0) { try {
localStorage.setItem(LOCAL_STORAGE_PASSAGES, JSON.stringify(dbPassages)); } catch
(e) {} return dbPassages; }
try { const localData = localStorage.getItem(LOCAL_STORAGE_PASSAGES); return localData
? JSON.parse(localData) : []; } catch (e) { return []; } }, async savePassage(passage) { try { const
currentList = await this.getAllPassages(); const idx = currentList.findIndex(p => p.id ===
passage.id); let newList = idx >= 0 ? [...currentList] : [passage, ...currentList]; if (idx >= 0)
newList[idx] = passage; localStorage.setItem(LOCAL_STORAGE_PASSAGES,
JSON.stringify(newList)); } catch (e) {}
try { const db = await this.open(); await new Promise((resolve, reject) => { const tx =
db.transaction('passages', 'readwrite'); const store = tx.objectStore('passages'); const req =
store.put(passage); req.onsuccess = () => resolve(passage); req.onerror = (err) => reject(err);
}); } catch (e) {}
```

```
if (window.CherryFirebase && window.CherryFirebase.auth?.currentUser) { try { const { db, appId, doc, setDoc } = window.CherryFirebase; const userId = window.CherryFirebase.auth.currentUser.uid; const ref = doc(db, 'artifacts', appId, 'users', userId, 'passages', passage.id); await setDoc(ref, { ...passage, updatedAt: Date.now() }, { merge: true }); } catch (e) {} } return passage; }, async deletePassage(id) { try { const localData = localStorage.getItem(LOCAL_STORAGE_PASSAGES); if (localData) { const list = JSON.parse(localData).filter(p => p.id !== id); localStorage.setItem(LOCAL_STORAGE_PASSAGES, JSON.stringify(list)); } } catch (e) {} try { const db = await this.open(); await new Promise((resolve, reject) => { const tx = db.transaction('passages', 'readwrite'); const store = tx.objectStore('passages'); const req = store.delete(id); req.onsuccess = () => resolve(true); req.onerror = (err) => reject(err); }); } catch (e) {}
if (window.CherryFirebase && window.CherryFirebase.auth?.currentUser) { try { const { db, appId, doc, deleteDoc } = window.CherryFirebase; const userId = window.CherryFirebase.auth.currentUser.uid; const ref = doc(db, 'artifacts', appId, 'users', userId, 'passages', id); await deleteDoc(ref); } catch (e) {} } return true; }, async fetchCloudPassages(user) { if (!window.CherryFirebase || !user) return null; try { const { db, appId, collection, getDocs } = window.CherryFirebase; const ref = collection(db, 'artifacts', appId, 'users', user.uid, 'passages'); const snap = await getDocs(ref); const cloudPassages = []; snap.forEach(d => cloudPassages.push(d.data())); return cloudPassages; } catch (e) { return null; } }, async getSettings() { try { const db = await this.open(); const res = await new Promise((resolve, reject) => { const tx = db.transaction('settings', 'readonly'); const store = tx.objectStore('settings'); const req = store.get('app_settings'); req.onsuccess = () => resolve(req.result?.value || null); req.onerror = (err) => reject(err); }); if (res) return res; } catch (e) {}
try { const local = localStorage.getItem(LOCAL_STORAGE_SETTINGS); return local ? JSON.parse(local) : null; } catch (e) { return null; } }, async saveSettings(settings) { try { localStorage.setItem(LOCAL_STORAGE_SETTINGS, JSON.stringify(settings)); } catch (e) {} try { const db = await this.open(); await new Promise((resolve, reject) => { const tx = db.transaction('settings', 'readwrite'); const store = tx.objectStore('settings'); const req = store.put({ key: 'app_settings', value: settings }); req.onsuccess = () => resolve(true); req.onerror = (err) => reject(err); }); } catch (e) {} return true; } };
// ========================================= // 2. Sound & TTS Synthesizer Helpers // ========================================= function playSound(type) { try { const ctx = new (window.AudioContext || window.webkitAudioContext)(); const osc = ctx.createOscillator(); const gain = ctx.createGain(); osc.connect(gain); gain.connect(ctx.destination);
if (type === 'correct') { osc.type = 'sine'; osc.frequency.setValueAtTime(523.25, ctx.currentTime); osc.frequency.setValueAtTime(659.25, ctx.currentTime + 0.1); osc.frequency.setValueAtTime(783.99, ctx.currentTime + 0.2); gain.gain.setValueAtTime(0.2, ctx.currentTime); gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.4); osc.start(); osc.stop(ctx.currentTime + 0.4); } else if (type === 'wrong') { osc.type = 'sawtooth'; osc.frequency.setValueAtTime(220, ctx.currentTime); osc.frequency.setValueAtTime(180, ctx.currentTime + 0.15); gain.gain.setValueAtTime(0.25, ctx.currentTime); gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.35); osc.start(); osc.stop(ctx.currentTime + 0.35); } } catch (e) {} }
function speakText(text, rate = 0.85) { if (!('speechSynthesis' in window)) return; window.speechSynthesis.cancel(); const utterance = new SpeechSynthesisUtterance(text);
```

```
utterance.lang = 'en-US'; utterance.rate = parseFloat(rate);
window.speechSynthesis.speak(utterance); }
// Fallback Translation Engine async function translateToKorean(text) { if (!text || !text.trim())
return ''; try { const url =
https://translate.googleapis.com/translate_a/single?client=gtx&sl=en&tl=ko&dt=t&q=${encodeU
RIComponent(text.trim())}; const res = await fetch(url); const data = await res.json(); if (data &&
data[0]) { return data[0].map(item => item[0]).join(''); } } catch (e) {} return (한글 해석 입력 필요)
${text}; }
// Tokenization function tokenizeSentence(text) { if (!text) return []; const rawWords =
text.trim().split(/\s+/); return rawWords.map((w, idx) => { const match = w.match(/^([^a-zA-Z0-9
% −] +)? (. *?)([^a − zA − Z0 − 9%-]+)?$/); if (!match) { return { id: tok_${idx}_${Date.now()},
raw: w, word: w, prefix: '', suffix: '' }; } return { id: tok_${idx}_${Date.now()}, raw: w, word:
match[2] || w, prefix: match[1] || '', suffix: match[3] || '' }; }); }
// Concept SVG Generator function generateConceptSvg(sentences) { const width = 760; const
height = Math.max(240, sentences.length * 80 + 40); const nodesSvg = sentences.map((sent, i)
=> { const y = 50 + i * 80; return <g class="concept-node" cursor="pointer"> <rect x="60" y="
y − 25"width = "640"height = "50"rx = "14"fill = "#ffffff"stroke = "#e11d48"stroke − width = "2"cla
{y + 5}" font-size="20">sent. cue||'📌' < /text >< textx = "125"y = "{y - 3}" font-size="14"
font-weight="bold" fill="#1c1e24">
sent. en. length > 35? sent. en. substring(0, 32) + '...': sent. en < /text >< textx = "125"y = "
{y + 15}" font-size="12" fill="#e11d48">${(sent.ko || '').substring(0, 45)}</text> </g> ${i <
sentences.length - 1 ? <line x1="380" y1="${y + 25}" x2="380" y2="
y + 55"stroke = "#cbd5e1"stroke − width = "3"stroke − dasharray = "4"/ >< polygonpoints = "375,
{y + 53} 385,y + 53380,{y + 60}" fill="#94a3b8" /> : ''}; }).join('');
return <svg viewBox="0 0 ${width} ${height}" xmlns="http://www.w3.org/2000/svg" class="w-full
h-auto"> <rect width="100%" height="100%" fill="#fff1f2" rx="20" class="dark:fill-stone-900" />
${nodesSvg} </svg>; }
// ========== // 3. Main React App //
========== function App() { const [passages,
setPassages] = useState([]); const [currentPassage, setCurrentPassage] = useState(null); const
[activeTab, setActiveTab] = useState('home'); const [userPoints, setUserPoints] = useState(0);
const [settings, setSettings] = useState({ apiKey: '', blankWidthMode: 'proportional',
stage3Reveal: false, blankRatio: '100', ttsRate: '0.85', theme: 'light' }); const [toast, setToast] =
useState(null); const [highlightSentenceId, setHighlightSentenceId] = useState(null);
// Auth & PWA States const [user, setUser] = useState(null); const [isAuthModalOpen,
setIsAuthModalOpen] = useState(false); const [deferredInstallPrompt, setDeferredInstallPrompt]
= useState(null); const [isInstalledApp, setIsInstalledApp] = useState(false);
useEffect(() => { async function loadData() { const loadedPassages = await
CherryDB.getAllPassages(); setPassages(loadedPassages);
const loadedSettings = await CherryDB.getSettings(); if (loadedSettings) {
setSettings(loadedSettings); if (loadedSettings.theme === 'dark')
document.documentElement.classList.add('dark'); }
const savedCurrentId = localStorage.getItem(LOCAL_STORAGE_CURRENT_ID); if
(savedCurrentId) { const found = loadedPassages.find(p => p.id === savedCurrentId); if (found)
setCurrentPassage(found); }
const savedTab = localStorage.getItem(LOCAL_STORAGE_TAB); if (savedTab)
setActiveTab(savedTab); } loadData(); }, []);
```

```
useEffect(() => { const handleBeforeInstall = (e) => { e.preventDefault();
setDeferredInstallPrompt(e); }; window.addEventListener('beforeinstallprompt',
handleBeforeInstall); if (window.matchMedia('(display-mode: standalone)').matches ||
window.navigator.standalone) { setIsInstalledApp(true); } return () =>
window.removeEventListener('beforeinstallprompt', handleBeforeInstall); }, []);
useEffect(() => { const checkLocalUser = () => { try { const stored =
localStorage.getItem('cherry_current_user'); if (stored) return JSON.parse(stored); } catch(e){}
return null; };
const handleLocalUserChange = (e) => setUser(e.detail || null);
window.addEventListener('cherry_user_changed', handleLocalUserChange);
if (!window.CherryFirebase || !window.CherryFirebase.auth) { setUser(checkLocalUser()); return
() => window.removeEventListener('cherry_user_changed', handleLocalUserChange); }
const { auth, onAuthStateChanged } = window.CherryFirebase; const unsubscribe =
onAuthStateChanged(auth, async (currentUser) => { if (currentUser &&
!currentUser.isAnonymous) { setUser(currentUser); const cloudData = await
CherryDB.fetchCloudPassages(currentUser); if (cloudData && cloudData.length > 0) {
setPassages(cloudData); for (const p of cloudData) await CherryDB.savePassage(p); } } else {
setUser(checkLocalUser()); } });
return () => { unsubscribe && unsubscribe();
window.removeEventListener('cherry_user_changed', handleLocalUserChange); }; }, []);
const duePassages = useMemo(() => { const now = Date.now(); return passages.filter(p => { if
(!p.lastStudiedAt) return true; return ((now - p.lastStudiedAt) / (1000 * 60 * 60 * 24)) >= 1; }); },
[passages]);
const showToast = (message, type = 'info', undoCallback = null) => { setToast({ message, type,
undoCallback }); if (!undoCallback) setTimeout(() => setToast(null), 3500); };
const switchTab = (tab) => { setActiveTab(tab); try {
localStorage.setItem(LOCAL_STORAGE_TAB, tab); } catch (e) {} };
const handleSavePassage = async (passage) => { await CherryDB.savePassage(passage);
setPassages(prev => { const idx = prev.findIndex(p => p.id === passage.id); if (idx >= 0) { const
copy = [...prev]; copy[idx] = passage; return copy; } return [passage, ...prev]; });
setCurrentPassage(passage); try { localStorage.setItem(LOCAL_STORAGE_CURRENT_ID,
passage.id); } catch (e) {} switchTab('stage1'); showToast("지문이 성공적으로 저장되었습니다!
🍒"); };
const handleDeletePassage = (id) => { const target = passages.find(p => p.id === id); if (!target)
return;
setPassages(prev => prev.filter(p => p.id !== id)); if (currentPassage && currentPassage.id ===
id) { setCurrentPassage(null); switchTab('home'); }
let timerId = setTimeout(async () => { await CherryDB.deletePassage(id); }, 5000);
showToast("${target.title}" 지문이 삭제되었습니다., 'warning', async () => {
clearTimeout(timerId); setPassages(prev => [target, ...prev]); await
CherryDB.savePassage(target); setToast(null); showToast("지문 삭제가 취소되어
복원되었습니다. 🍒"); }); };
const handleCompleteStage = (completedStage) => { setUserPoints(prev => prev + 10);
showToast(축하합니다! ${completedStage === 'stage2' ? '2단계' : '3단계'} 완료! (+10 🍒)); if
(currentPassage) { const updated = { ...currentPassage, progress: { ...currentPassage.progress,
[completedStage]: true }, lastStudiedAt: Date.now() }; handleSavePassage(updated); } };
return ( <div className="min-h-screen bg-stone-50 dark:bg-darkbg text-stone-900
dark:text-stone-100 flex flex-col"> {/* Header */} <header className="sticky top-0 z-40
```

bg-white/90 dark:bg-darkcard/90 backdrop-blur-md border-b border-stone-200 dark:border-stone-800 px-4 py-3"> <div className="flex items-center gap-3 cursor-pointer" onClick={() => switchTab('home')}> 🍒   <h1 className="font-black text-lg tracking-tight leading-none text-stone-900 dark:text-stone-100">체리</h1> 영어 지문 통암기
  🍒 {userPoints} P
{deferredInstallPrompt && !isInstalledApp && (<button onClick={() => { deferredInstallPrompt.prompt(); deferredInstallPrompt.userChoice.then(choice => { if (choice.outcome === 'accepted') setIsInstalledApp(true); setDeferredInstallPrompt(null); }); }} className="px-3 py-1.5 bg-rose-50 text-rose-600 dark:bg-rose-950/40 dark:text-rose-300 font-black text-xs rounded-xl hover:bg-rose-100 transition-all" > 📱 앱 설치 </button>)}
<button onClick={() => setIsAuthModalOpen(true)} className="px-3 py-1.5 bg-stone-100 dark:bg-stone-800 hover:bg-stone-200 text-stone-700 dark:text-stone-200 rounded-xl font-bold text-xs transition-all flex items-center gap-1.5" > {user && !user.isAnonymous ? (user.email ? user.email.split('@')[0] : '내 계정') : '로그인'} </button>
<button onClick={() => switchTab('settings')} className="w-9 h-9 bg-stone-100 dark:bg-stone-800 hover:bg-stone-200 text-stone-700 dark:text-stone-200 rounded-xl flex items-center justify-center text-xs transition-all" > </button>
<button onClick={() => switchTab('create')} className="px-3.5 py-2 bg-cherry-600 hover:bg-cherry-700 text-white rounded-xl font-black text-xs shadow-md transition-all active:scale-95 flex items-center gap-1.5" > 지문 추가 </button>   </header>
{/* Content Area */} <main className="flex-1 max-w-5xl w-full mx-auto p-4 sm:p-6"> {activeTab === 'home' && (<HomeView passages={passages} duePassages={duePassages} user={user} onSelectPassage={(p, tab = 'stage1') => { setCurrentPassage(p); try { localStorage.setItem(LOCAL_STORAGE_CURRENT_ID, p.id); } catch(e){} switchTab(tab); }} onDeletePassage={handleDeletePassage} onExport={() => { const dataStr = "data:text/json;charset=utf-8," + encodeURIComponent(JSON.stringify(passages, null, 2)); const dl = document.createElement('a'); dl.setAttribute("href", dataStr); dl.setAttribute("download", cherry_backup_${new Date().toISOString().slice(0,10)}.json); dl.click(); showToast("지문 데이터가 JSON 파일로 백업되었습니다.", "success"); }} onImport={(e) => { const file = e.target.files[0]; if (!file) return; const reader = new FileReader(); reader.onload = async (evt) => { try { const imported = JSON.parse(evt.target.result); if (Array.isArray(imported)) { for (const p of imported) await handleSavePassage(p); showToast(${imported.length}개 지문 복원 성공!, "success"); } } catch (err) { showToast("JSON 형식이 올바르지 않습니다.", "wrong"); } }; reader.readAsText(file); }} />)}
{activeTab === 'create' && (<CreatePassageView settings={settings} onSavePassage={handleSavePassage} onCancel={() => switchTab('home')} />)}
{activeTab === 'stage1' && currentPassage && (<Stage1View passage={currentPassage} settings={settings} highlightSentenceId={highlightSentenceId}

```
onUpdatePassage={handleSavePassage} onNavigateStage={(st) => switchTab(st)}
onBackHome={() => switchTab('home')} onOpenVisual={() => switchTab('visual')}
onOpenPrint={() => switchTab('print')} /> )}
{activeTab === 'stage2' && currentPassage && ( <Stage2View passage={currentPassage}
settings={settings} onUpdatePassage={handleSavePassage} onCompleteStage={() =>
handleCompleteStage('stage2')} onBackHome={() => switchTab('home')} /> )}
{activeTab === 'stage3' && currentPassage && ( <Stage3View passage={currentPassage}
settings={settings} onUpdatePassage={handleSavePassage} onCompleteStage={() =>
handleCompleteStage('stage3')} onBackHome={() => switchTab('home')} /> )}
{activeTab === 'visual' && currentPassage && ( <VisualSummaryView
passage={currentPassage} onBackHome={() => switchTab('stage1')} /> )}
{activeTab === 'print' && currentPassage && ( <PrintView passage={currentPassage}
onBack={() => switchTab('stage1')} /> )}
{activeTab === 'settings' && ( <SettingsView settings={settings} onSave={async (newSettings)
=> { setSettings(newSettings); await CherryDB.saveSettings(newSettings); if
(newSettings.theme === 'dark') document.documentElement.classList.add('dark'); else
document.documentElement.classList.remove('dark'); showToast("설정이 저장되었습니다.",
"success"); switchTab('home'); }} onClose={() => switchTab('home')} /> )} </main>
{/* Auth Modal */} {isAuthModalOpen && ( <AuthModal user={user} onClose={() =>
setIsAuthModalOpen(false)} onAuthSuccess={(msg) => { setIsAuthModalOpen(false);
showToast(msg, "success"); }} /> )}
{/* Toast Snackbar */} {toast && ( <div className="fixed bottom-6 right-6 z-50 flex items-center
gap-3 bg-stone-900 dark:bg-stone-100 text-white dark:text-stone-900 px-5 py-3.5 rounded-2xl
shadow-2xl font-bold text-sm"> <span>{toast.message}</span> {toast.undoCallback && (
<button onClick={toast.undoCallback} className="ml-2 px-3 py-1 bg-cherry-600 text-white
rounded-xl text-xs font-black">실행 취소 </button> )} </div> )} </div> ); }
// ========================================== // HOME VIEW //
========================================== function HomeView({ passages,
duePassages, user, onSelectPassage, onDeletePassage, onExport, onImport }) { const [search,
setSearch] = useState(''); const [sort, setSort] = useState('recent');
const filteredPassages = useMemo(() => { let list = [...passages]; if (search.trim()) { const q =
search.toLowerCase(); list = list.filter(p => p.title.toLowerCase().includes(q) ||
p.sentences.some(s => s.en.toLowerCase().includes(q) || s.ko.includes(q))); } if (sort ===
'recent') list.sort((a, b) => (b.lastStudiedAt || 0) - (a.lastStudiedAt || 0)); else if (sort === 'created')
list.sort((a, b) => (b.createdAt || 0) - (a.createdAt || 0)); else if (sort === 'title') list.sort((a, b) =>
a.title.localeCompare(b.title)); return list; }, [passages, search, sort]);
return ( <div className="space-y-6 pb-12"> {duePassages && duePassages.length > 0 && (
<div className="p-4 sm:p-5 bg-gradient-to-r from-cherry-500 to-rose-600 text-white
rounded-3xl shadow-lg flex flex-col sm:flex-row items-start sm:items-center justify-between
gap-3"> <div className="flex items-center gap-3"> <div className="w-11 h-11 rounded-2xl
bg-white/20 flex items-center justify-center text-2xl shrink-0">⏰</div> <div> <h3
className="font-black text-sm sm:text-base">오늘 복습 지문 {duePassages.length}개</h3> <p
className="text-xs text-white/80">에빙하우스 주기 복습으로 장기 기억으로 전환하세요!</p>
</div> </div> <button onClick={() => onSelectPassage(duePassages[0], 'stage2')}
className="w-full sm:w-auto px-5 py-3 bg-white text-cherry-700 font-black text-xs rounded-2xl
shadow hover:bg-stone-100" > 오늘 복습 시작하기 🍒 </button> </div> )}
<div className="flex flex-col sm:flex-row gap-3 items-center justify-between"> <div
className="relative w-full sm:w-72"> <i className="fa-solid fa-magnifying-glass absolute
```

left-3.5 top-1/2 -translate-y-1/2 text-stone-400 text-xs"> <input type="text" placeholder="지문 제목 또는 내용 검색..." value={search} onChange={(e) => setSearch(e.target.value)} className="w-full pl-9 pr-4 py-3 bg-white dark:bg-darkcard border border-stone-200 dark:border-stone-800 rounded-2xl font-bold text-xs focus:outline-none focus:border-cherry-500 shadow-sm" />
 <select value={sort} onChange={(e) => setSort(e.target.value)} className="px-3 py-3 bg-white dark:bg-darkcard border border-stone-200 dark:border-stone-800 rounded-xl font-bold text-xs text-stone-700 dark:text-stone-300 shadow-sm" > <option value="recent">최근 학습순</option> <option value="created">최신 등록순</option> <option value="title">제목 가나다순</option> </select> <button onClick={onExport} className="px-3.5 py-3 bg-white dark:bg-darkcard border border-stone-200 dark:border-stone-800 rounded-xl text-xs font-bold text-stone-700 dark:text-stone-300 shadow-sm hover:bg-stone-50"> 백업 </button>
<label className="px-3.5 py-3 bg-white dark:bg-darkcard border border-stone-200 dark:border-stone-800 rounded-xl text-xs font-bold text-stone-700 dark:text-stone-300 shadow-sm hover:bg-stone-50 cursor-pointer"> 복원 <input type="file" accept=".json" onChange={onImport} className="hidden" /> </label>
{filteredPassages.length === 0 ? (🍒 <h3 className="font-black text-lg text-stone-900 dark:text-stone-100">등록된 지문이 없습니다</h3> 상단의 [+ 지문 추가] 버튼을 눌러 새로운 영어 지문을 등록하거나 교재 사진을 올리세요!   ) : (
{filteredPassages.map((passage) => (<h3 className="font-black text-base text-stone-900 dark:text-stone-100 line-clamp-1 group-hover:text-cherry-600"> {passage.title} </h3> <button onClick={(e) => { e.stopPropagation(); onDeletePassage(passage.id); }} className="text-stone-300 hover:text-rose-500 p-1.5 text-xs"> </button>
 {passage.sentences[0]?.en}
  문장 {passage.sentences.length}개   {passage.memorizeTarget === 'original' ? '원문 암기' : 'AI 요약 암기'}
 <button onClick={() => onSelectPassage(passage, 'stage1')} className="flex-1 py-3 bg-stone-100 dark:bg-stone-800 hover:bg-cherry-50 hover:text-cherry-600 text-stone-700 dark:text-stone-300 rounded-xl text-xs font-black"> 1단계 읽기 </button> <button onClick={() => onSelectPassage(passage, 'stage2')} className="flex-1

```
py-3 bg-stone-100 dark:bg-stone-800 hover:bg-cherry-50 hover:text-cherry-600 text-stone-700 dark:text-stone-300 rounded-xl text-xs font-black"> 2단계 단어뱅크 </button> <button onClick={() => onSelectPassage(passage, 'stage3')} className="flex-1 py-3 bg-cherry-600 hover:bg-cherry-700 text-white rounded-xl text-xs font-black shadow"> 3단계 입력 </button> </div> </div> ))} </div> )} </div> ); }
// ========================================== // AUTH MODAL // ========================================== function AuthModal({ user, onClose, onAuthSuccess }) { const [isSignUp, setIsSignUp] = useState(false); const [email, setEmail] = useState(''); const [password, setPassword] = useState(''); const [error, setError] = useState(''); const [loading, setLoading] = useState(false);
const handleAuth = async (e) => { e.preventDefault(); setError(''); if (!email || !password) return setError("이메일과 비밀번호를 모두 입력하세요."); if (password.length < 6) return setError("비밀번호는 6자리 이상이어야 합니다.");
setLoading(true); try { if (window.CherryFirebase && window.CherryFirebase.auth) { const { auth, signInWithEmailAndPassword, createUserWithEmailAndPassword } = window.CherryFirebase; try { if (isSignUp) { await createUserWithEmailAndPassword(auth, email, password); onAuthSuccess("회원가입 성공 및 로그인 되었습니다! 🍒"); setLoading(false); return; } else { await signInWithEmailAndPassword(auth, email, password); onAuthSuccess("로그인 성공! 클라우드 동기화 활성화."); setLoading(false); return; } } catch (fbErr) { if (fbErr.code === 'auth/email-already-in-use') setError("이미 가입된 이메일 주소입니다."); else if (fbErr.code === 'auth/wrong-password' || fbErr.code === 'auth/user-not-found' || fbErr.code === 'auth/invalid-credential') setError("이메일 또는 비밀번호가 올바르지 않습니다."); else setError(fbErr.message); setLoading(false); return; } }
// Local fallback account engine const localUsersKey = 'cherry_local_users'; const localUsers = JSON.parse(localStorage.getItem(localUsersKey) || '{}'); if (isSignUp) { if (localUsers[email]) { setError("이미 가입된 이메일 주소입니다."); setLoading(false); return; } const newUser = { uid: 'local_' + Date.now(), email, password }; localUsers[email] = newUser; localStorage.setItem(localUsersKey, JSON.stringify(localUsers)); localStorage.setItem('cherry_current_user', JSON.stringify(newUser)); window.dispatchEvent(new CustomEvent('cherry_user_changed', { detail: newUser })); onAuthSuccess("로컬 계정이 생성되어 로그인되었습니다! 🍒"); } else { const existing = localUsers[email]; if (!existing || existing.password !== password) { setError("이메일 또는 비밀번호가 올바르지 않습니다."); setLoading(false); return; } localStorage.setItem('cherry_current_user', JSON.stringify(existing)); window.dispatchEvent(new CustomEvent('cherry_user_changed', { detail: existing })); onAuthSuccess("로그인 성공!"); } } catch (err) { setError("인증 과정에서 오류가 발생했습니다."); } finally { setLoading(false); } };
return ( <div className="fixed inset-0 z-50 bg-black/60 backdrop-blur-sm flex items-center justify-center p-4"> <div className="bg-white dark:bg-darkcard rounded-3xl max-w-md w-full p-6 shadow-2xl space-y-4"> <div className="flex items-center justify-between border-b border-stone-100 dark:border-stone-800 pb-3"> <h3 className="font-black text-lg text-stone-900 dark:text-stone-100"> {user && !user.isAnonymous ? '내 계정 관리' : isSignUp ? '체리 계정 만들기' : '계정 로그인'} </h3> <button onClick={onClose} className="text-stone-400 hover:text-stone-600 font-bold text-xs p-1"> <i className="fa-solid fa-xmark text-base"></i> </button> </div>
{user && !user.isAnonymous ? ( <div className="space-y-4"> <div className="p-4 bg-stone-50 dark:bg-stone-800/60 rounded-2xl space-y-1"> <span className="text-[11px] font-bold text-stone-400">로그인 계정</span> <p className="font-black text-sm
```

```
text-cherry-600 truncate">{user.email}</p> </div> <button onClick={() => { if (window.CherryFirebase?.auth) window.CherryFirebase.signOut(window.CherryFirebase.auth); localStorage.removeItem('cherry_current_user'); window.dispatchEvent(new CustomEvent('cherry_user_changed', { detail: null })); onAuthSuccess("로그아웃 되었습니다."); }} className="w-full py-3.5 bg-stone-100 dark:bg-stone-800 hover:bg-stone-200 text-stone-700 dark:text-stone-200 font-black text-xs rounded-xl" > 로그아웃 </button> </div> ) : ( <form onSubmit={handleAuth} className="space-y-3"> {error && <div className="p-3 bg-rose-50 text-rose-600 rounded-xl text-xs font-bold">{error}</div>} <div> <label className="block text-[11px] font-bold text-stone-500 mb-1">이메일 주소</label> <input type="email" required placeholder="example@cherry.com" value={email} onChange={(e) => setEmail(e.target.value)} className="w-full p-3 bg-stone-50 dark:bg-stone-800 border border-stone-200 dark:border-stone-700 rounded-xl font-bold text-xs focus:outline-none focus:border-cherry-500" /> </div> <div> <label className="block text-[11px] font-bold text-stone-500 mb-1">비밀번호 (6자리 이상)</label> <input type="password" required placeholder="••••••••" value={password} onChange={(e) => setPassword(e.target.value)} className="w-full p-3 bg-stone-50 dark:bg-stone-800 border border-stone-200 dark:border-stone-700 rounded-xl font-bold text-xs focus:outline-none focus:border-cherry-500" /> </div> <button type="submit" disabled={loading} className="w-full py-3.5 bg-cherry-600 hover:bg-cherry-700 text-white font-black text-xs rounded-xl shadow-lg" > {loading ? '처리 중...' : isSignUp ? '회원가입 완료' : '로그인하기'} </button> <div className="text-center pt-1"> <button type="button" onClick={() => setIsSignUp(!isSignUp)} className="text-[11px] font-bold text-stone-400 hover:text-cherry-600" > {isSignUp ? '이미 계정이 있나요? 로그인' : '계정이 없으신가요? 회원가입'} </button> </div> </form> )} </div> </div> ); }
// ========================================== // CREATE PASSAGE VIEW // ========================================== function CreatePassageView({ settings, onSavePassage, onCancel }) { const [mode, setMode] = useState('summary'); const [lengthMode, setLengthMode] = useState('medium'); const [title, setTitle] = useState(''); const [rawText, setRawText] = useState(''); const [loading, setLoading] = useState(false); const [previewSentences, setPreviewSentences] = useState(null);
const handleGenerateSummary = async () => { if (!rawText.trim()) return; setLoading(true);
try { let sentences = []; let generatedTitle = title.trim() || '영어 지문 ' + new Date().toLocaleDateString();
if (settings.apiKey) { const systemPrompt = Summarize text into logical sentences according to length:
lengthMode. RespondONLYwithvalidJSON: "title": string, "sentences": ["en": string, "ko": string, "cue": string];
{settings.apiKey}; const res = await fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ contents: [{ parts: [{ text: rawText }] }], systemInstruction: { parts: [{ text: systemPrompt }] }, generationConfig: { responseMimeType: "application/json" } }) }); const jsonRes = await res.json(); const parsed = JSON.parse(jsonRes.candidates[0].content.parts[0].text); generatedTitle = title || parsed.title || generatedTitle; sentences = parsed.sentences; } else { const rawSplit = rawText.split(/(?<=[.!?])\s+/).filter(s => s.trim().length > 0); const cues = ['📌', '💡', '🚀', '⭐', '🔍', '🎯']; for (let i = 0; i < rawSplit.length; i++) { const enSent = rawSplit[i].trim(); const koSent = await translateToKorean(enSent); sentences.push({ id: s_${i}_${Date.now()}, en: enSent, ko: koSent, cue: cues[i % cues.length] }); } }
for (let s of sentences) { if (!s.ko || s.ko === s.en) s.ko = await translateToKorean(s.en); if (!s.id) s.id = sent_${Math.random().toString(36).substr(2, 9)}; }
setPreviewSentences({ title: generatedTitle, sentences }); } catch (e) { console.error(e); } finally {
```

```
setLoading(false); } };
const handleFinalSave = () => { if (!previewSentences) return; const visualSvg =
generateConceptSvg(previewSentences.sentences); const passageObj = { id: p_${Date.now()},
title: previewSentences.title, originalText: rawText, memorizeTarget: mode, sentences:
previewSentences.sentences, visualSummary: { svg: visualSvg, stale: false }, createdAt:
Date.now(), lastStudiedAt: Date.now(), progress: { stage1: false, stage2: false, stage3: false },
annotations: { strokes: [], memos: {} } }; onSavePassage(passageObj); };
return ( <div className="max-w-3xl mx-auto space-y-6"> <div className="flex items-center
justify-between"> <h2 className="font-black text-2xl">지문 등록 및 번역 생성</h2> <button
onClick={onCancel} className="text-xs font-bold text-stone-400">취소</button> </div>
{!previewSentences ? ( <div className="bg-white dark:bg-darkcard p-6 rounded-3xl border
border-stone-200 dark:border-stone-800 shadow-sm space-y-4"> <div> <label
className="block text-xs font-bold text-stone-500 mb-1">지문 제목 (선택)</label> <input
type="text" placeholder="예: 수능특강 15강 2번" value={title} onChange={(e) =>
setTitle(e.target.value)} className="w-full p-3 bg-stone-50 dark:bg-stone-800 border
border-stone-200 dark:border-stone-700 rounded-xl font-bold text-sm" /> </div>
<div> <label className="block text-xs font-bold text-stone-500 mb-1">영어 지문 원문</label>
<textarea rows="7" placeholder="외우고자 하는 영어 지문 원문을 입력하거나 붙여넣으세요..."
value={rawText} onChange={(e) => setRawText(e.target.value)} className="w-full p-4
bg-stone-50 dark:bg-stone-800 border border-stone-200 dark:border-stone-700 rounded-2xl
font-medium text-sm leading-relaxed" ></textarea> </div>
<button onClick={handleGenerateSummary} disabled={loading || !rawText.trim()}
className="w-full py-4 bg-cherry-600 hover:bg-cherry-700 text-white font-black text-sm
rounded-2xl shadow-lg disabled:opacity-50" > {loading ? 'AI 분석 및 한글 해석 생성 중...' : '요약
및 한글 해석 완성하기 🍒'} </button> </div> ) : ( <div className="bg-white dark:bg-darkcard
p-6 rounded-3xl border border-stone-200 dark:border-stone-800 shadow-sm space-y-5"> <h3
className="font-black text-lg">요약 및 한글 해석 확인</h3>
{previewSentences.sentences.map((sent, idx) => ( <div key={idx} className="p-4 bg-stone-50
dark:bg-stone-800/50 rounded-2xl space-y-2"> <input type="text" value={sent.en}
onChange={(e) => { const copy = [...previewSentences.sentences]; copy[idx].en =
e.target.value; setPreviewSentences({ ...previewSentences, sentences: copy }); }}
className="w-full p-2.5 bg-white dark:bg-darkcard border rounded-xl font-bold text-sm" />
<input type="text" value={sent.ko} onChange={(e) => { const copy =
[...previewSentences.sentences]; copy[idx].ko = e.target.value; setPreviewSentences({
...previewSentences, sentences: copy }); }} className="w-full p-2.5 bg-white dark:bg-darkcard
border rounded-xl font-medium text-xs text-cherry-600 dark:text-cherry-400" /> </div> ))}
<button onClick={handleFinalSave} className="w-full py-4 bg-cherry-600 hover:bg-cherry-700
text-white font-black text-base rounded-2xl shadow-xl" > 저장하고 1단계 학습 시작하기 🚀
</button> </div> )} </div> ); }
// ========================================== // STAGE 1 VIEW (Reading +
Drawing Canvas) // ========================================== function
Stage1View({ passage, settings, highlightSentenceId, onUpdatePassage, onNavigateStage,
onBackHome, onOpenVisual, onOpenPrint }) { const [hideKo, setHideKo] = useState(false);
const [penMode, setPenMode] = useState(false); const [penColor, setPenColor] =
useState('#e11d48'); const [strokes, setStrokes] = useState(passage.annotations?.strokes || []);
const [isDrawing, setIsDrawing] = useState(false); const canvasRef = useRef(null);
useEffect(() => { const canvas = canvasRef.current; if (!canvas) return; const ctx =
canvas.getContext('2d'); const rect = canvas.getBoundingClientRect(); canvas.width =
```

```
rect.width; canvas.height = rect.height;
ctx.clearRect(0, 0, canvas.width, canvas.height); strokes.forEach(s => { if (s.points.length < 2)
return; ctx.beginPath(); ctx.strokeStyle = s.color; ctx.lineWidth = 3; ctx.lineCap = 'round';
ctx.moveTo(s.points[0].x * canvas.width, s.points[0].y * canvas.height); for (let i = 1; i <
s.points.length; i++) { ctx.lineTo(s.points[i].x * canvas.width, s.points[i].y * canvas.height); }
ctx.stroke(); }); }, [strokes, penMode]);
const handlePointerDown = (e) => { if (!penMode) return; const canvas = canvasRef.current;
const rect = canvas.getBoundingClientRect(); const x = (e.clientX - rect.left) / rect.width; const y
= (e.clientY - rect.top) / rect.height; setIsDrawing(true); setStrokes(prev => [...prev, { color:
penColor, points: [{ x, y }] }]); };
const handlePointerMove = (e) => { if (!isDrawing || !penMode) return; const canvas =
canvasRef.current; const rect = canvas.getBoundingClientRect(); const x = (e.clientX - rect.left) /
rect.width; const y = (e.clientY - rect.top) / rect.height; setStrokes(prev => { const copy =
[...prev]; copy[copy.length - 1]?.points.push({ x, y }); return copy; }); };
return ( <div className="space-y-6 relative"> <div className="bg-white dark:bg-darkcard p-4
rounded-3xl border border-stone-200 dark:border-stone-800 flex flex-wrap items-center
justify-between gap-3 shadow-sm"> <div className="flex items-center gap-2"> <button
onClick={onBackHome} className="text-stone-400 font-bold text-sm"> <i className="fa-solid
fa-arrow-left"></i> 홈 </button> <span className="text-stone-300">|</span> <h2
className="font-black text-lg">{passage.title}</h2> </div>
<div className="flex items-center gap-2"> <button onClick={() => setHideKo(!hideKo)}
className={px-3 py-1.5 rounded-xl text-xs font-bold border ${hideKo ? 'bg-cherry-600
text-white' : 'bg-stone-100 text-stone-700'}} > 해석 가리기 {hideKo ? 'ON' : 'OFF'} </button>
<button onClick={() => setPenMode(!penMode)} className={px-3 py-1.5 rounded-xl text-xs
font-bold border ${penMode ? 'bg-cherry-600 text-white' : 'bg-stone-100 text-stone-700'}} > ✏️
필기 {penMode ? 'ON' : 'OFF'} </button> <button onClick={onOpenVisual} className="px-3
py-1.5 bg-rose-50 text-rose-700 border rounded-xl text-xs font-bold"> 🎨 개념도 </button>
<button onClick={onOpenPrint} className="px-3 py-1.5 bg-stone-100 text-stone-700 border
rounded-xl text-xs font-bold"> 🖨️ 인쇄 </button> </div> </div>
<div className="relative bg-white dark:bg-darkcard p-6 rounded-3xl border border-stone-200
dark:border-stone-800 shadow-sm space-y-6"> <canvas ref={canvasRef}
onPointerDown={handlePointerDown} onPointerMove={handlePointerMove} onPointerUp={() =>
{ setIsDrawing(false); onUpdatePassage({ ...passage, annotations: { ...passage.annotations,
strokes } }); }} className={absolute inset-0 z-10 w-full h-full touch-canvas ${penMode ?
'cursor-crosshair' : 'pointer-events-none'}} />
{passage.sentences.map((sent, idx) => ( <div key={sent.id} className="p-4 rounded-2xl
bg-stone-50 dark:bg-stone-800/40 border border-stone-200 dark:border-stone-800 space-y-1">
<div className="flex items-center justify-between"> <span className="text-xs font-black
text-cherry-600">#{idx + 1} {sent.cue}</span> <button onClick={() => speakText(sent.en,
settings.ttsRate)} className="w-7 h-7 rounded-lg bg-stone-200 text-xs"> <i
className="fa-solid fa-volume-high"></i> </button> </div> <p className="font-bold text-base
text-stone-900 dark:text-stone-100 leading-relaxed">{sent.en}</p> <p className={font-medium
text-sm text-cherry-600 dark:text-cherry-400 ${hideKo ? 'blur-sm' : ''}}> {sent.ko || '(한글 해석
없음)'} </p> </div> ))}
<div className="pt-4 border-t flex justify-end"> <button onClick={() =>
onNavigateStage('stage2')} className="px-6 py-3 bg-cherry-600 text-white font-black text-sm
rounded-2xl shadow-lg"> 2단계 단어 뱅크 훈련 🚀 </button> </div> </div> </div> ); }
// ========================================== // STAGE 2 VIEW //
```

```
========== function Stage2View({ passage, settings, onUpdatePassage, onCompleteStage, onBackHome }) { const [currentIndex, setCurrentIndex] = useState(0); const [slots, setSlots] = useState([]); const [bankChips, setBankChips] = useState([]); const [isGraded, setIsGraded] = useState(false); const [isCorrect, setIsCorrect] = useState(false);
const currentSentence = passage.sentences[currentIndex]; const tokens = useMemo(() => tokenizeSentence(currentSentence?.en), [currentSentence]);
useEffect(() => { if (!tokens.length) return; const initialChips = tokens.map((t, idx) => ({ id: t.id, word: t.word, originalBankIndex: idx })); const shuffled = [...initialChips]; for (let i = shuffled.length - 1; i > 0; i--) { const j = Math.floor(Math.random() * (i + 1)); [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]]; } setBankChips(shuffled); setSlots(new Array(tokens.length).fill(null)); setIsGraded(false); setIsCorrect(false); }, [currentIndex, currentSentence]);
const handleChipClick = (chip) => { if (isGraded) return; const firstEmpty = slots.findIndex(s => s === null); if (firstEmpty === -1) return; const newSlots = [...slots]; newSlots[firstEmpty] = chip; setSlots(newSlots); setBankChips(prev => prev.filter(c => c.id !== chip.id)); };
const handleSlotClick = (slotIndex) => { if (isGraded) return; const targetChip = slots[slotIndex]; if (!targetChip) return; const newBank = [...bankChips]; newBank.splice(targetChip.originalBankIndex, 0, targetChip); setBankChips(newBank); const newSlots = [...slots]; newSlots[slotIndex] = null; setSlots(newSlots); };
const handleGrade = () => { let correctCount = 0; slots.forEach((s, i) => { if (s && s.word.toLowerCase() === tokens[i].word.toLowerCase()) correctCount++; }); const allCorrect = correctCount === tokens.length; setIsGraded(true); setIsCorrect(allCorrect); if (allCorrect) { playSound('correct'); setTimeout(() => { if (currentIndex < passage.sentences.length - 1) setCurrentIndex(prev => prev + 1); else onCompleteStage(); }, 1200); } else { playSound('wrong'); } };
return ( <div className="max-w-3xl mx-auto space-y-6"> <div className="bg-white dark:bg-darkcard p-4 rounded-3xl border flex justify-between"> <button onClick={onBackHome} className="text-stone-400 font-bold text-sm"><i className="fa-solid fa-arrow-left"></i> 홈</button> <span className="text-xs font-black text-cherry-600">문장 {currentIndex + 1} / {passage.sentences.length}</span> </div>
<div className="bg-white dark:bg-darkcard p-6 rounded-3xl border space-y-6"> <div className="p-4 bg-cherry-50 rounded-2xl border"> <span className="text-[11px] font-bold text-cherry-600">한글 해석</span> <p className="font-bold text-base mt-1">{currentSentence?.ko}</p> </div>
<div className="flex flex-wrap items-center gap-2 min-h-[90px] p-4 bg-stone-50 rounded-2xl border"> {tokens.map((tok, idx) => { const filled = slots[idx]; return ( <div key={idx} className="flex items-center gap-0.5"> {tok.prefix && <span className="font-bold">{tok.prefix}</span>} <button onClick={() => handleSlotClick(idx)} className={min-w-[50px] h-10 px-3 rounded-xl font-bold text-sm border-2 ${ filled ? (isGraded ? (filled.word.toLowerCase() === tok.word.toLowerCase() ? 'bg-emerald-100 border-emerald-500 text-emerald-800' : 'bg-rose-100 border-rose-500 text-rose-800') : 'bg-white border-cherry-500') : 'bg-stone-200 border-dashed' }} > {filled ? filled.word : ''} </button> {tok.suffix && <span className="font-bold">{tok.suffix}</span>} </div> ); })} </div>
<div className="p-4 bg-stone-100 rounded-2xl space-y-2"> <span className="text-xs font-bold text-stone-500">단어 뱅크</span> <div className="flex flex-wrap gap-2"> {bankChips.map(chip => ( <button key={chip.id} onClick={() => handleChipClick(chip)} className="min-h-[44px] px-4 py-2.5 bg-white border hover:border-cherry-500 rounded-xl font-bold text-sm shadow-sm" > {chip.word} </button> ))} </div> </div>
```

```
<button onClick={handleGrade} disabled={slots.some(s => s === null)} className="w-full
py-3.5 bg-cherry-600 hover:bg-cherry-700 text-white font-black text-sm rounded-2xl shadow-lg
disabled:opacity-40" > 채점하기 🍒 </button> </div> </div> ); }
// ========================================== // STAGE 3 VIEW //
========================================== function Stage3View({ passage,
settings, onUpdatePassage, onCompleteStage, onBackHome }) { const [currentIndex,
setCurrentIndex] = useState(0); const [userInputs, setUserInputs] = useState([]); const
[isGraded, setIsGraded] = useState(false); const [isCorrect, setIsCorrect] = useState(false);
const currentSentence = passage.sentences[currentIndex]; const tokens = useMemo(() =>
tokenizeSentence(currentSentence?.en), [currentSentence]); const inputRefs = useRef([]);
useEffect(() => { if (!tokens.length) return; setUserInputs(new Array(tokens.length).fill(''));
setIsGraded(false); setIsCorrect(false); setTimeout(() => { if (inputRefs.current[0])
inputRefs.current[0].focus(); }, 100); }, [currentIndex, currentSentence]);
const handleGrade = () => { let correctCount = 0; userInputs.forEach((val, i) => { if
(val.trim().toLowerCase() === tokens[i].word.toLowerCase()) correctCount++; }); const
allCorrect = correctCount === tokens.length; setIsGraded(true); setIsCorrect(allCorrect);
if (allCorrect) { playSound('correct'); setTimeout(() => { if (currentIndex <
passage.sentences.length - 1) setCurrentIndex(prev => prev + 1); else onCompleteStage(); },
1200); } else { playSound('wrong'); } };
return ( <div className="max-w-3xl mx-auto space-y-6"> <div className="bg-white
dark:bg-darkcard p-4 rounded-3xl border flex justify-between"> <button onClick={onBackHome}
className="text-stone-400 font-bold text-sm"><i className="fa-solid fa-arrow-left"></i>
홈</button> <span className="text-xs font-black text-cherry-600">3단계 - 직접 입력</span>
</div>
<div className="bg-white dark:bg-darkcard p-6 rounded-3xl border space-y-6"> <div
className="p-4 bg-cherry-50 rounded-2xl border"> <span className="text-[11px] font-bold
text-cherry-600">한글 해석</span> <p className="font-bold text-base
mt-1">{currentSentence?.ko}</p> </div>
<div className="flex flex-wrap items-center gap-2 min-h-[90px] p-4 bg-stone-50 rounded-2xl
border"> {tokens.map((tok, idx) => ( <div key={idx} className="flex items-center gap-0.5">
{tok.prefix && <span className="font-bold">{tok.prefix}</span>} <input ref={el =>
inputRefs.current[idx] = el} type="text" inputMode="text" enterKeyHint={idx === tokens.length -
1 ? "done" : "next"} autoCapitalize="none" autoCorrect="off" autoComplete="off"
spellCheck="false" value={userInputs[idx] || ''} onChange={(e) => { const copy = [...userInputs];
copy[idx] = e.target.value; setUserInputs(copy); }} onKeyDown={(e) => { if (e.key === ' ' || e.key
=== 'Enter') { e.preventDefault(); if (idx === tokens.length - 1) handleGrade(); else if
(inputRefs.current[idx + 1]) inputRefs.current[idx + 1].focus(); } else if (e.key === 'Backspace' &&
!userInputs[idx] && idx > 0) { e.preventDefault(); if (inputRefs.current[idx - 1])
inputRefs.current[idx - 1].focus(); } }} className={h-11 px-2 text-center rounded-xl font-bold
text-sm border-2 ${ isGraded ? userInputs[idx]?.trim().toLowerCase() ===
tok.word.toLowerCase() ? 'bg-emerald-100 border-emerald-500' : 'bg-rose-100 border-rose-500'
: 'bg-white border-stone-300' }} style={{ width: ${Math.max(48, tok.word.length * 12 + 18)}px }} />
{tok.suffix && <span className="font-bold">{tok.suffix}</span>} </div> ))} </div>
{isGraded && !isCorrect && ( <div className="p-4 bg-rose-50 border border-rose-200
rounded-2xl"> <span className="text-xs font-bold text-rose-600">정답</span> <p
className="font-bold text-sm text-rose-900">{currentSentence.en}</p> </div> )}
<button onClick={handleGrade} className="w-full py-4 bg-cherry-600 text-white font-black
text-base rounded-2xl shadow-lg"> 채점하고 다음 문장 🚀 </button> </div> </div> ); }
```

```
// ========================================== // VISUAL SUMMARY & PRINT VIEWS // ========================================== function VisualSummaryView({ passage, onBackHome }) { return ( <div className="space-y-6"> <button onClick={onBackHome} className="text-stone-400 font-bold text-sm"><i className="fa-solid fa-arrow-left"></i> 돌아가기</button> <div className="bg-white p-6 rounded-3xl border shadow-sm" dangerouslySetInnerHTML={{ __html: passage.visualSummary?.svg || generateConceptSvg(passage.sentences) }} /> </div> ); } function PrintView({ passage, onBack }) { return ( <div className="space-y-6"> <div className="no-print flex justify-between"> <button onClick={onBack} className="text-stone-400 font-bold text-sm">돌아가기</button> <button onClick={() => window.print()} className="px-5 py-2.5 bg-cherry-600 text-white font-bold text-xs rounded-xl">🖨️ 브라우저 인쇄 / PDF 저장</button> </div> <div className="bg-white p-8 rounded-3xl border space-y-4"> <h1 className="text-2xl font-black text-center pb-4 border-b">{passage.title} - 암기 시트</h1> {passage.sentences.map((s, i) => ( <div key={i} className="pb-2 border-b space-y-1"> <p className="font-bold text-base">{i + 1}. {s.en}</p> <p className="text-xs text-stone-600">{s.ko}</p> </div> ))} </div> </div> ); } // ========================================== // SETTINGS VIEW // ========================================== function SettingsView({ settings, onSave, onClose }) { const [apiKey, setApiKey] = useState(settings.apiKey || ''); const [theme, setTheme] = useState(settings.theme || 'light');
return ( <div className="max-w-xl mx-auto space-y-6"> <div className="flex justify-between items-center"> <h2 className="font-black text-2xl">설정 (Settings)</h2> <button onClick={onClose} className="text-xs font-bold text-stone-400">닫기</button> </div>
<div className="bg-white dark:bg-darkcard p-6 rounded-3xl border shadow-sm space-y-4">
<div> <label className="block text-xs font-bold text-stone-500 mb-1">Gemini API 키 (선택)</label> <input type="password" placeholder="AI 요약 및 자동 추출용 키..." value={apiKey} onChange={(e) => setApiKey(e.target.value)} className="w-full p-3 bg-stone-50 border rounded-xl font-bold text-sm" /> </div>
<div> <label className="block text-xs font-bold text-stone-500 mb-1">화면 테마 설정</label> <select value={theme} onChange={(e) => setTheme(e.target.value)} className="w-full p-3 bg-stone-50 border rounded-xl font-bold text-xs"> <option value="light">라이트 모드</option> <option value="dark">다크 모드</option> </select> </div>
<button onClick={() => onSave({ ...settings, apiKey, theme })} className="w-full py-4 bg-cherry-600 text-white font-black text-sm rounded-2xl shadow-lg">설정 저장 🍒 </button> </div> </div> ); }
// Render App const root = ReactDOM.createRoot(document.getElementById('root')); root.render(<App />); </script> </body> </html>
```